

API + JSON + FLASK + POSTMAN

Duración: 50 minutos | Trabajo en equipos de 3 como máximo estudiantes

Propósito:

Reconocer las características de las APIs y del formato JSON, comprender la función de Postman y relacionar estas herramientas mediante la creación y prueba de una API sencilla con Flask.

Al finalizar la actividad podrás:

- Distinguir las características básicas de una API.
- Explicar las características del formato JSON y su relación con servicios Web y APIs.
- Identificar las características y funciones principales de Postman.
- Relacionar Postman con el desarrollo y prueba de APIs.
- Levantar y probar una API sencilla desarrollada con Flask.

Organización de la sesión

Actividad	Tema	Tiempo	Producto
1	API y JSON	15 min	Clasificación y explicación
2	Flask: levantar una API	20 min	API funcionando + evidencia
3	Postman: probar la API	15 min	Pruebas + evidencia

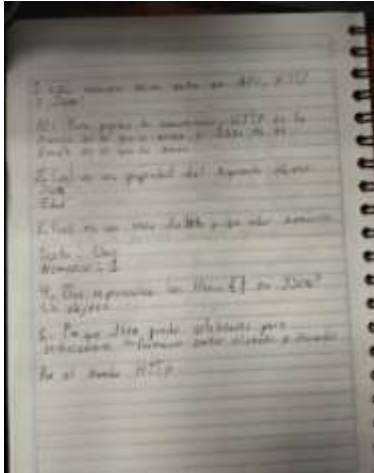
ACTIVIDAD 1. ¿API, JSON o comunicación cliente-servidor?

Tiempo: 15 minutos

En equipo, analicen y clasifiquen los siguientes conceptos en tres categorías: A) API, B) JSON y C) comunicación cliente-servidor.

API · Endpoint · Cliente · Servidor · HTTP · GET · POST · JSON · Servicio Web · Recurso · Request · Response · Postman – Recurso – REST- PUT- DELETE- Cliente-Servidor-URL-HTTPS status code-Header-BodyObjeto-Propiedad- Valor-Array-Number-String-Boolean-Null-Servicio Web- Flask-HTML-CSS-PYTHON

Después, respondan en su hoja de trabajo:



Reto JSON:

```
{  
  "nombre": "Juan",  
  "edad": 18,  
  "carrera": "Programación"  
}
```

Producto: Clasificación de conceptos y respuestas del reto JSON.

ACTIVIDAD 2. Levanta tu primera API con Flask- Todos los integrantes de manera colaborativa

Tiempo: 20 minutos

Objetivo: levantar un servidor Flask y obtener respuestas en formato JSON.

1. Crea una carpeta llamada:

api_flask/

2. Dentro de la carpeta crea un archivo llamado:

app.py

3. Instala Flask desde la terminal:

pip install flask

4. Escribe el siguiente código en app.py:

```
from flask import Flask, jsonify
```

```
app = Flask(__name__)
```

```
@app.route("/")  
def inicio():
```

```

return jsonify({
    "mensaje": "API funcionando",
    "curso": "Desarrollo de APIs"
})

```

```

@app.route("/alumnos")
def alumnos():
    return jsonify([
        {
            "id": 1,
            "nombre": "Ana"
        },
        {
            "id": 2,
            "nombre": "Carlos"
        }
    ])

```

```

if __name__ == "__main__":
    app.run(debug=True)

```

5. Ejecuta el servidor:

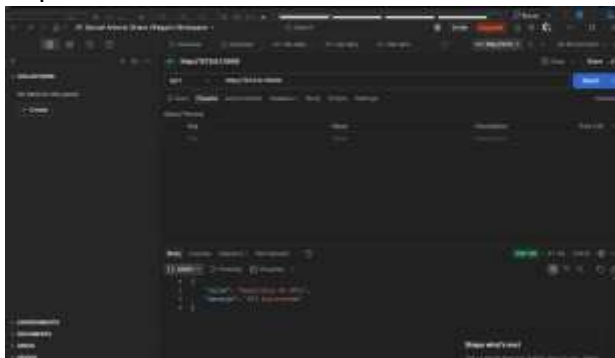
python app.py

```

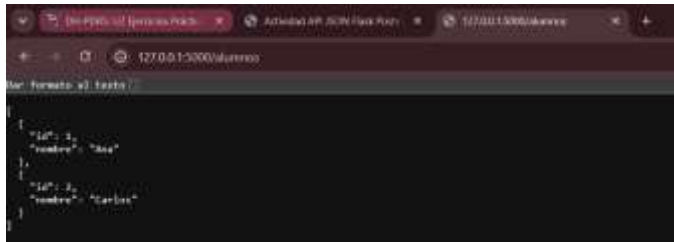
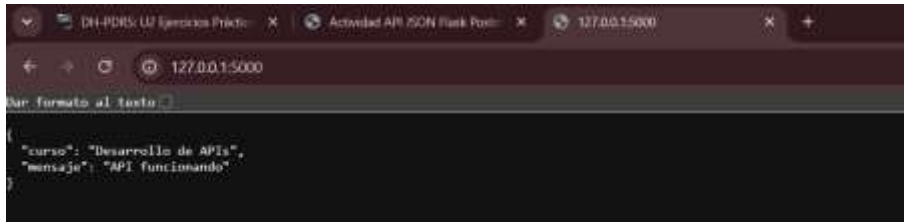
C:\Users\jrodriguez> python app.py
Output from Flask: 0.12.1
Requirement already satisfied: Jinja2<3.0, in C:\Users\jrodriguez\AppData\Local\Python\Python39-64\Lib\site-packages
from Flask 0.12.1
Requirement already satisfied: Werkzeug<2.0, in C:\Users\jrodriguez\AppData\Local\Python\Python39-64\Lib\site-packages
from Flask 0.12.1
Requirement already satisfied: MarkupSafe<2.0, in C:\Users\jrodriguez\AppData\Local\Python\Python39-64\Lib\site-packages
from Flask 0.12.1
Output from Flask: 0.12.1
Warning: Werkzeug is not installed, but it is required for the debugger.
C:\Users\jrodriguez\AppData\Local\Python\Python39-64\Scripts> python app.py
File "C:\Users\jrodriguez\AppData\Local\Python\Python39-64\Scripts\python.exe", line 1
from flask import Flask, jsonify
^
SyntaxError: invalid syntax
C:\Users\jrodriguez\AppData\Local\Python\Python39-64\Scripts> python app.py
* Serving Flask app "app"
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL-C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 123-456-789
127.0.0.1 - - [02/Oct/2024 12:40:00] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [02/Oct/2024 12:40:00] "GET /flask/app HTTP/1.1" 200 -
127.0.0.1 - - [02/Oct/2024 12:40:00] "GET / HTTP/1.1" 200 -

```

6. Verifica en la terminal que Flask indique que el servidor está disponible, por ejemplo:
http://127.0.0.1:5000

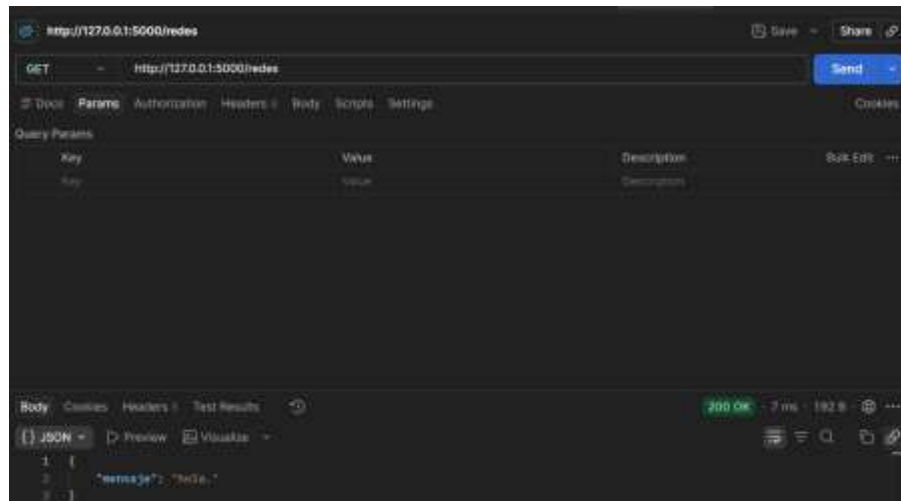


7. Prueba en el navegador: http://127.0.0.1:5000/ http://127.0.0.1:5000/alumnos



Responde:

6. Agrega una nueva ruta /redes donde salude



7. ¿Cuál es el endpoint /alumnos?

es una ruta dentro de la API que responde a las peticiones HTTP GET enviando un listado de alumnos en formato JSON.

8. ¿Qué método HTTP estamos utilizando?

El método GET

9. ¿Qué información devuelve el endpoint /alumnos y el endpoint de redes?

El endpoint devuelve los datos de los alumnos id y nombre y el endpoint de redes recibe el saludo

Los datos de los alumnos y el id

Producto: Captura de pantalla donde se observe el servidor Flask funcionando y una respuesta JSON.

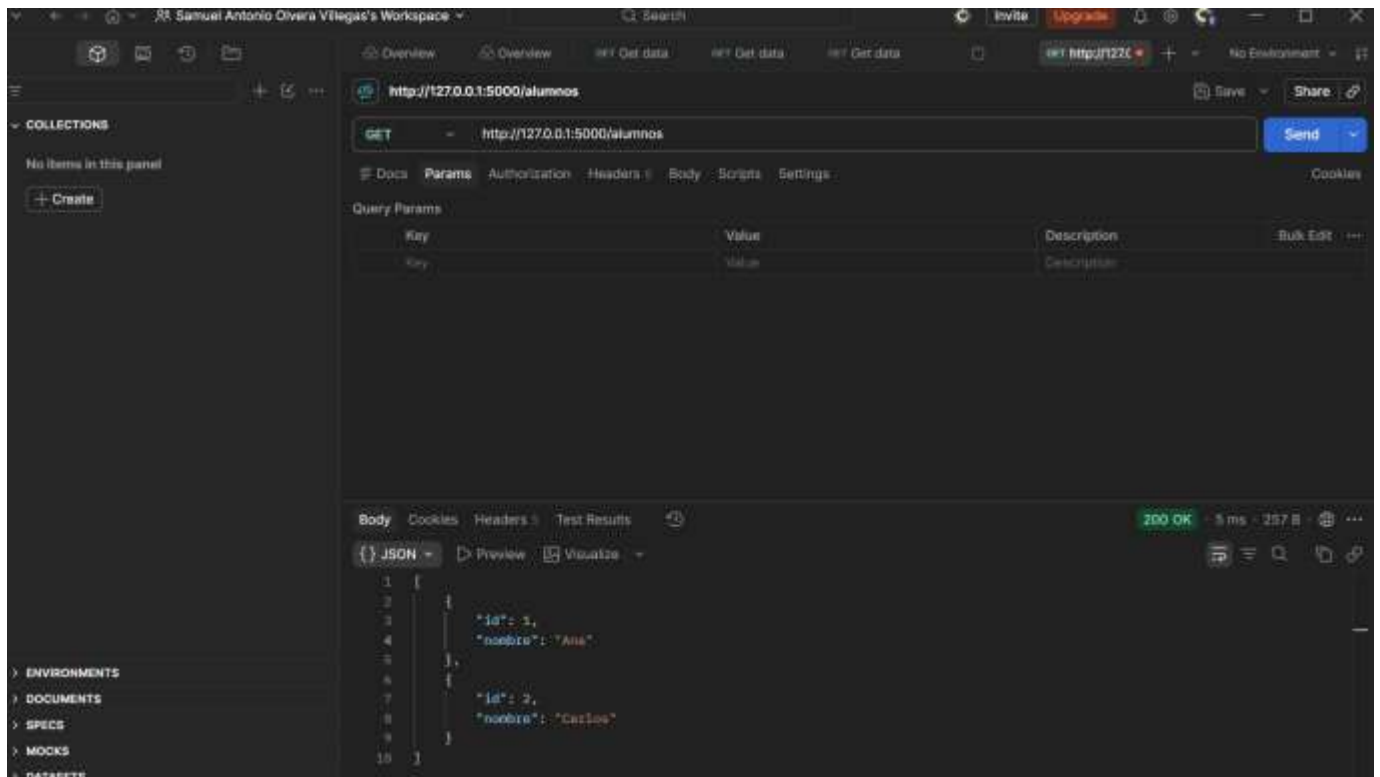


ACTIVIDAD 3. Prueba tu API con Postman

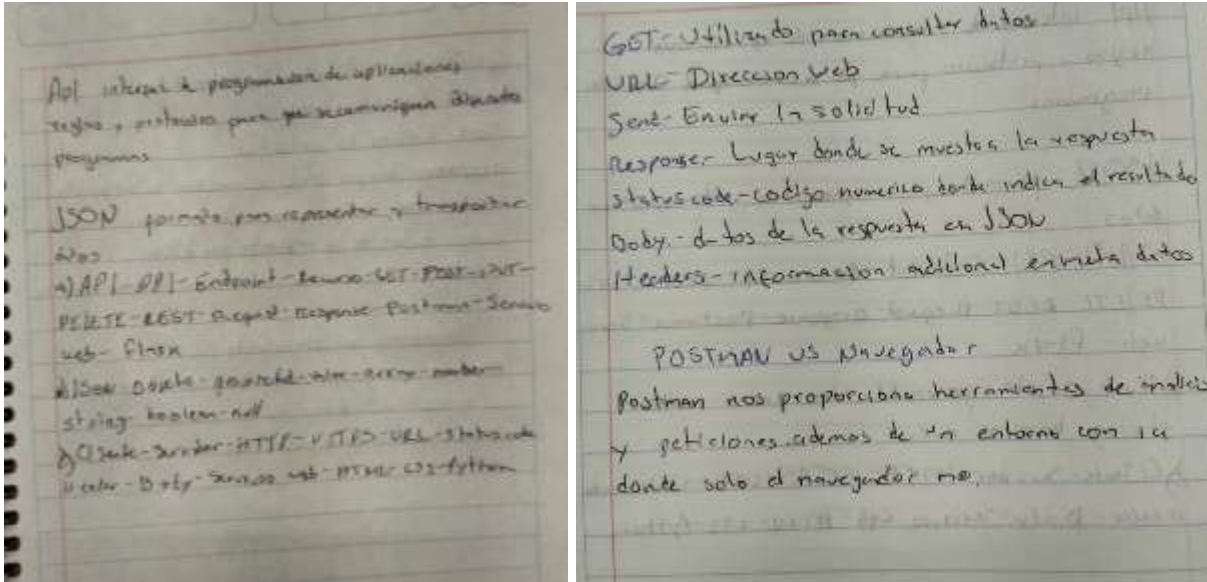
Tiempo: 15 minutos

Objetivo: utilizar Postman para enviar solicitudes a la API creada con Flask y analizar sus respuestas.

10. Abre Postman.
11. Crea una nueva petición utilizando el método GET.
12. Escribe la URL: `http://127.0.0.1:5000/`
13. Presiona Send y observa la respuesta.
14. Crea una segunda petición GET con la URL: `http://127.0.0.1:5000/alumnos`
15. Presiona Send y analiza la respuesta JSON.



Identifica en Postman:

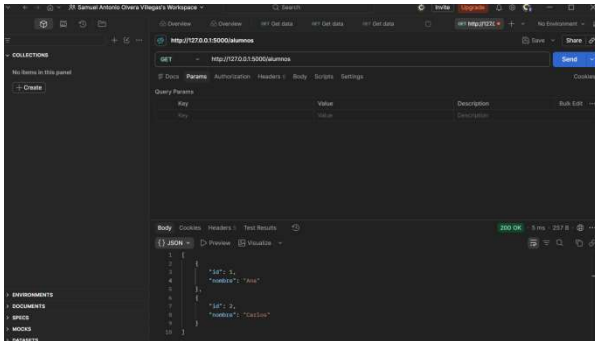


Reto final:

Explica con tus propias palabras: ¿qué ventaja tiene Postman respecto a probar una API solamente desde el navegador?

Te da mas detalles de lo que te daría un navegador

Producto: Captura de pantalla de Postman donde aparezca GET /alumnos, el código 200 OK y la respuesta JSON.



CIERRE. El API en 60 segundos

Tiempo: 5 minutos

Participa en la puesta en común. El docente podrá seleccionar estudiantes o equipos para responder:

- ¿Qué es una API y para qué sirve?
- ¿Qué es JSON y por qué se utiliza en las APIs?
- ¿Para qué sirve Postman en el desarrollo de una API?
- ¿Qué relación existe entre Flask y Postman?

Idea clave: Flask crea y ejecuta la API → Postman envía peticiones → Flask responde → Postman permite inspeccionar la respuesta.

Evaluación de la sesión

Actividad	Evidencia	Puntos
1. API + JSON	Clasificación + explicación	2
2. Flask	API funcionando	5
3. Postman	Peticiones y respuestas	3
Total		10

Roles sugeridos dentro del equipo

Rol	Responsabilidad
Analista	Resuelve y explica los conceptos de API y JSON.
Desarrollador	Levanta y verifica la API con Flask.
Tester	Realiza las pruebas con Postman.
Documentador	Registra respuestas y evidencias.

Importante: cambien los roles durante la sesión para que todos los integrantes participen.